

Light Local LLM System - Master Implementation Guide

Enterprise-Grade Local AI System with Distributed LLM, RAG, MCP, LSP, and Multi-Agent Support

Version: 1.0
Last Updated: 2026-04-04
Target: AI Coding Agents Implementation

Executive Summary

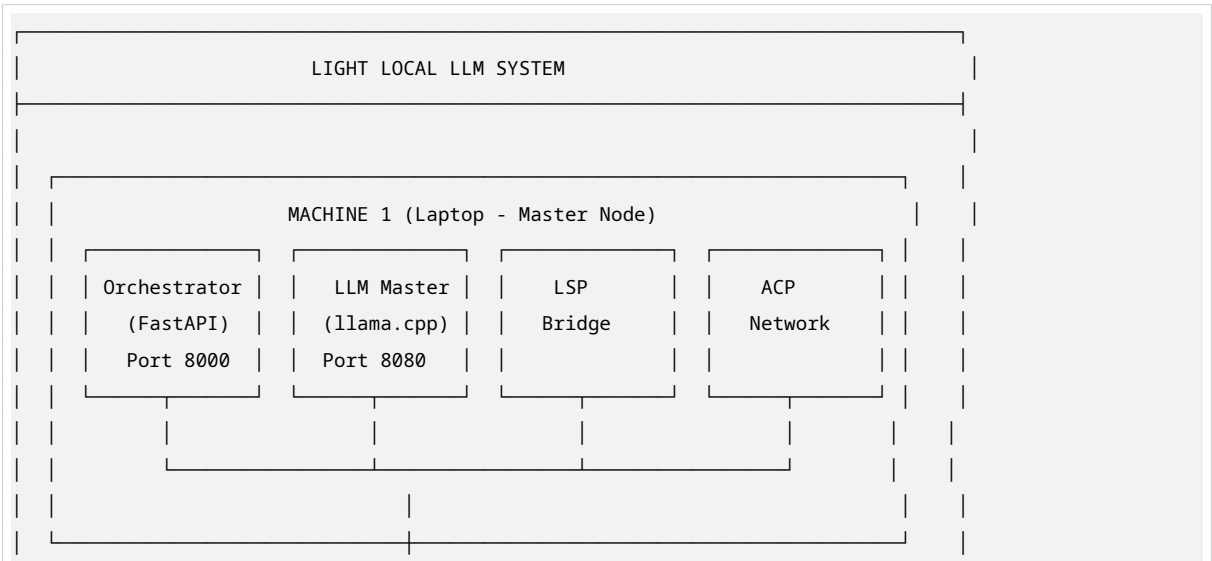
This master implementation guide provides a comprehensive, step-by-step blueprint for building an enterprise-grade local AI system that combines:

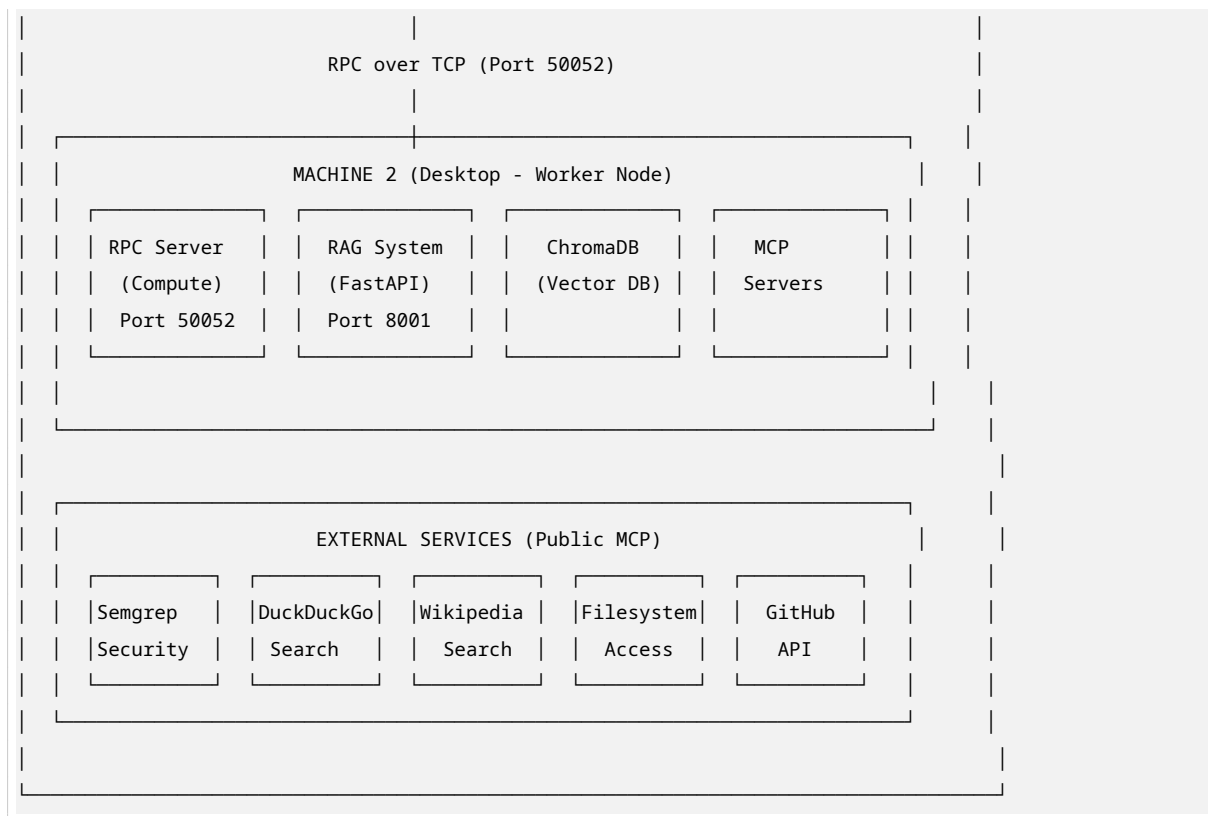
- **Distributed LLM** via llama.cpp with RPC clustering
- **RAG System** with enterprise software knowledge base
- **MCP Integration** for tool ecosystem access
- **LSP Integration** for code intelligence
- **Orchestrator Layer** for coordination
- **ACP Multi-Agent** system for collaboration

Hardware Configuration

Machine	Role	Specifications
Machine 1 (Master)	Primary Node	Lenovo ThinkBook 16 Pro, AMD Ryzen 9, 32GB DDR4, NVIDIA RTX, 4TB NVMe
Machine 2 (Worker)	Knowledge Server	Intel i7 11th Gen, 64GB DDR4, 2x 2TB NVMe
Network	Internal	Wired Ethernet 1GbE

System Architecture Overview





Implementation Phases

Phase Overview

Phase	Component	Priority	Est. Time	Dependencies
1	Infrastructure & RPC Cluster	HIGH	2-4 hours	None
2	RAG System	HIGH	4-6 hours	Phase 1 (partial)
3	MCP Integration	HIGH	3-5 hours	None
4	LSP Integration	HIGH	3-4 hours	None
5	Orchestrator Layer	HIGH	6-8 hours	Phases 1-4
6	ACP Multi-Agent	MEDIUM	4-6 hours	Phase 5
7	Testing & Deployment	MEDIUM	3-4 hours	All Phases

Phase 1: Infrastructure & RPC Cluster Setup

File: phase1_infrastructure_rpc_setup.md

Purpose: Set up distributed LLM using llama.cpp with RPC clustering across both machines

Key Components

1. Network Configuration

- Static IP assignment (Machine 1: 192.168.1.100, Machine 2: 192.168.1.101)
- Windows Firewall rules for ports 50052 (RPC) and 8080 (API)
- Network connectivity verification

2. llama.cpp Installation

- Download pre-built Windows binaries (CUDA 12.4)
- Extract to C:\llama on both machines
- Environment PATH configuration

3. RPC Worker Node (Desktop - Machine 2)

```
.\rpc-server.exe -p 50052 -H 0.0.0.0 -c
```

4. RPC Master Node (Laptop - Machine 1)

```
.\llama-server.exe -m "C:\models\Llama-3.1-70B.Q4_K_M.gguf" -c 8192 -ngl 99 --rpc  
↪ 192.168.1.101:50052
```

5. Recommended Models

- **Primary:** Llama-3.1-70B-Instruct (Q4_K_M, ~38GB) - BEST CHOICE
- **Alternative:** Qwen2.5-14B-Instruct (Q4_K_M, ~9GB)
- **Maximum:** Mixtral-8x22B-Instruct (Q4_K_M, ~67GB)

Verification Steps

1. Check GPU utilization: `nvidia-smi`
2. Test API endpoint: `curl http://localhost:8080/v1/models`
3. Verify RPC connection in logs
4. Run inference benchmark

Phase 2: RAG System with Enterprise Knowledge Base

File: phase2_rag_system.md

Purpose: Build enterprise-grade RAG system for software development knowledge

Key Components

1. Embedding Model

- Primary: sentence-transformers/all-mpnet-base-v2 (768d, 420MB)
- Alternative: BAAI/bge-large-en-v1.5 (1024d, 1.3GB)
- Quantization support for <200MB RAM usage

2. Vector Database

- ChromaDB with persistent storage
- HNSW index optimization
- Collection management

3. Document Processing Pipeline

- Multi-format support (PDF, Markdown, Code files)
- Chunking: 1000 chars with 200 overlap
- Metadata extraction (category, source, hash)

4. Enterprise Knowledge Base Structure

```
docs/
├─ architecture/      (Clean Arch, DDD, Microservices, CQRS)
├─ backend/           (Spring Boot, .NET Core, Node.js, Python, Go)
├─ frontend/          (React, Angular, Vue, TypeScript)
├─ mobile/             (Flutter, React Native, Kotlin Multiplatform)
├─ desktop/            (Electron, Tauri, WPF)
├─ devops/             (Docker, Kubernetes, CI/CD)
├─ testing/            (Unit, Integration, E2E)
├─ security/           (OWASP, Best Practices)
└─ database/           (SQL, NoSQL, Migrations)
```

5. FastAPI Endpoints

- POST /query - Query knowledge base
- POST /ingest - Ingest documents
- GET /health - Health check
- GET /stats - KB statistics

Key Files Generated

- core/embeddings.py - Embedding model wrapper
- core/vector_store.py - ChromaDB operations
- core/document_processor.py - Document loading & chunking
- rag/pipeline.py - Main RAG pipeline
- api/main.py - FastAPI application

Phase 3: MCP Server Integration & Tool Ecosystem

File: phase3_mcp_integration.md
Purpose: Integrate Model Context Protocol for tool access

Key Components

1. Public MCP Servers

Server	URL	Purpose
Everything	everything.mcp.inevitable.fyi	Multi-purpose demo
Time	time.mcp.inevitable.fyi	Time functions
Semgrep	mcp.semgrep.ai/sse	Security scanning
DuckDuckGo	npx @mcp/duckduckgo	Web search
Wikipedia	npx @mcp/wikipedia	Knowledge base

Table 3 – continued

Server	URL	Purpose
Filesystem	<code>npx @modelcontextprotocol/server-filesystem</code>	File operations
GitHub	<code>npx @modelcontextprotocol/server-github</code>	GitHub API

2. MCP Client Implementation

- SSE and stdio transport support
- Automatic tool discovery
- Connection pooling
- Retry logic with exponential backoff

3. Orchestrator Integration

- Dynamic tool registration
- Tool selection logic
- Result processing

Key Files Generated

- `mcp_client.py` - Complete MCP client (25KB)
- `mcp_config.json` - Server configurations
- `orchestrator_integration.py` - Integration layer (20KB)
- `custom_server.py` - Custom server template (13KB)

Phase 4: LSP Integration for Code Intelligence

File: `phase4_lsp_integration.md`

Purpose: Integrate Language Server Protocol for code analysis

Key Components

1. Language Servers

Language	Server	Installation
TypeScript	<code>typescript-language-server</code>	<code>npm install -g typescript-language-server</code>
Python	<code>pyright</code>	<code>npm install -g pyright</code>
Rust	<code>rust-analyzer</code>	Built-in or standalone
Go	<code>gopls</code>	<code>go install golang.org/x/tools/gopls@latest</code>
Java	<code>jdtls</code>	Eclipse download

2. LSP-MCP Bridge

- LSP client wrapper

- MCP server interface
- Message translation layer
- Multi-server management

3. Code Intelligence Features

- Go to definition
- Find references
- Hover information
- Code completion
- Diagnostics
- Document/workspace symbols

Key Files Generated

- `lsp_mcp_bridge.py` - Complete bridge implementation (~900 lines)
- `lsp_config.json` - Multi-language configuration

Phase 5: Orchestrator Layer (Core Engine)

File: `phase5_orchestrator.md`

Purpose: Build central coordination layer

Key Components

1. FastAPI Application

- Lifecycle management
- Health checks
- CORS middleware
- Error handling

2. Agent Loop (ReAct Pattern)

User Query → RAG Context → LLM → Tool Decision → Tool Execution → Final Response

3. Integration Modules

- `llm_client.py` - llama.cpp integration
- `rag_client.py` - ChromaDB queries
- `mcp_manager.py` - Tool management
- `lsp_client.py` - Code intelligence

4. State Management

- Memory backend (default)
- Redis backend (optional)
- File backend (optional)
- Session TTL and history limits

5. API Endpoints

Endpoint	Method	Description
/chat	POST	Main chat endpoint
/chat/stream	POST	Streaming chat
/tools	GET	List available tools
/tools/execute	POST	Direct tool execution
/health	GET	Health check
/sessions	GET/DELETE	Session management

Key Files Generated

- app/main.py - FastAPI entry point
- app/config.py - Configuration management
- app/agent/loop.py - Agent loop
- app/integrations/*.py - Integration modules
- app/prompts/*.py - Prompt templates
- docker-compose.yml - Deployment config

Phase 6: ACP Multi-Agent Communication

File: phase6_acp_multiagent.md
Purpose: Enable multi-agent collaboration

Key Components

1. ACP Protocol

- Message-based communication
- JSON serialization
- Async/await throughout
- Request-response pattern

2. Agent Types

Agent	Role	Capabilities
MainOrchestrator	Central coordinator	Task routing, delegation
CodeAgent	Code operations	Analysis, generation, refactoring
ResearchAgent	Information gathering	Web search, summarization
ReviewAgent	Quality assurance	Code review, testing

3. Task Delegation

- Task creation and assignment
- Progress tracking
- Result aggregation
- Subtask support

4. Coordination Patterns

- Master-Worker
- Pipeline processing
- Voting/Consensus
- Peer-to-Peer

Key Files Generated

- `acp_core.py` - Core protocol classes
- `acp_registry.py` - Service discovery
- `acp_task_manager.py` - Task lifecycle
- `agents/*.py` - Specialized agents
- `config/agents.yaml` - Configuration

Phase 7: Testing, Monitoring & Deployment

File: `phase7_testing_monitoring_deployment.md`

Purpose: Production deployment and monitoring

Key Components

1. Docker Compose Stack

- 14 services with dependencies
- Health checks
- Network isolation
- Volume persistence

2. Monitoring Stack

Service	Port	Purpose
Prometheus	9090	Metrics collection
Grafana	3001	Visualization
Loki	3100	Log aggregation
Alertmanager	9093	Alert routing

3. Testing Suite

- Unit tests
- Integration tests
- Load tests (Locust)
- Benchmark scripts

4. Security Hardening

- Firewall configuration
- Docker security
- SSL/TLS certificates
- Secret management

5. Backup & Recovery

- Automated backups
- Component-specific backup
- Restore procedures

Key Files Generated

- docker/docker-compose.yml - Complete stack
 - monitoring/prometheus/prometheus.yml - Metrics config
 - monitoring/grafana/dashboards/*.json - Dashboards
 - tests/*.py - Test suites
 - scripts/deploy.sh - Deployment script
 - backup/backup.sh - Backup automation
-

Implementation Sequence for AI Coding Agents

Recommended Execution Order

```
Phase 1 (Infrastructure)
|
├─ Machine 2: Start RPC Server
|
├─ Machine 1: Start LLM Server with RPC
|
└─ Verify RPC connection

Phase 2 (RAG System - Machine 2)
|
├─ Install Python dependencies
├─ Setup ChromaDB
├─ Ingest enterprise documents
└─ Start RAG API server

Phase 3 (MCP Integration - Machine 1)
|
├─ Configure MCP servers
├─ Test MCP connections
└─ Verify tool discovery

Phase 4 (LSP Integration - Machine 1)
|
├─ Install language servers
├─ Configure LSP-MCP bridge
└─ Test code intelligence

Phase 5 (Orchestrator - Machine 1)
|
├─ Install dependencies
├─ Configure integrations
├─ Start orchestrator API
└─ Test end-to-end flow
```

Phase 6 (ACP Multi-Agent - Optional)

- |
- |— Setup ACP registry
- |— Deploy specialized agents
- |— Test agent collaboration

Phase 7 (Deployment)

- |
- |— Setup monitoring stack
- |— Configure backups
- |— Deploy to production
- |— Verify all health checks

Configuration Summary

Environment Variables

```
# LLM Configuration (Machine 1)
LLM_HOST=localhost
LLM_PORT=8080
LLM_MAX_TOKENS=4096
LLM_TEMPERATURE=0.7

# RAG Configuration (Machine 2)
RAG_HOST=192.168.1.101
RAG_PORT=8000
RAG_COLLECTION=documents
RAG_TOP_K=5

# MCP Configuration
MCP_SERVERS=https://everything.mcp.inevitable.fyi,https://mcp.semgrep.ai/sse
MCP_TIMEOUT=60

# LSP Configuration
LSP_ENABLED=true
LSP_SERVERS=/config/lsp_config.json

# State Management
STATE_BACKEND=memory
STATE_SESSION_TTL=3600
```

Network Ports

Port	Service	Machine
50052	RPC Server	Machine 2
8080	LLM API	Machine 1
8000	Orchestrator	Machine 1

Table 8 – continued

Port	Service	Machine
8001	RAG API	Machine 2
9090	Prometheus	Machine 1
3001	Grafana	Machine 1
3100	Loki	Machine 1

Testing Checklist

Phase 1: Infrastructure

- RPC server starts on Machine 2
- LLM server starts on Machine 1 with RPC flag
- Network connectivity verified
- GPU offloading confirmed
- Inference benchmark completed

Phase 2: RAG System

- Documents ingested successfully
- Embeddings generated
- Vector store persisted
- Query endpoint responds
- Search results relevant

Phase 3: MCP Integration

- Public MCP servers connected
- Tools discovered
- Tool execution works
- Error handling tested

Phase 4: LSP Integration

- Language servers installed
- LSP-MCP bridge running
- Code intelligence features work
- Multi-language support verified

Phase 5: Orchestrator

- API server starts
- Health check passes
- Chat endpoint works
- Tool calls execute
- RAG context injected
- Streaming responses work

Phase 6: ACP Multi-Agent

- Registry service running
- Agents register successfully
- Task delegation works
- Inter-agent communication verified

Phase 7: Deployment

- Docker Compose stack starts
- All health checks pass
- Monitoring dashboards accessible
- Alerts configured
- Backups automated

File Reference

Generated Documentation

File	Size	Description
phase1_infrastructure_rpc_setup.md	~1,150 lines	Infrastructure setup
phase2_rag_system.md	~1,994 lines	RAG implementation
phase3_mcp_integration.md	~93KB	MCP integration
phase4_lsp_integration.md	~900 lines	LSP integration
phase5_orchestrator.md	~4,121 lines	Orchestrator layer
phase6_acp_multiagent.md	~2,280 lines	Multi-agent system
phase7_testing_monitoring_deployment.md	~2,115 lines	Deployment guide

Generated Code Files

File	Size	Description
mcp_client.py	25KB	MCP client implementation
lsp_mcp_bridge.py	~900 lines	LSP-MCP bridge
orchestrator_integration.py	20KB	Orchestrator integration
custom_server.py	13KB	Custom MCP server template
mcp_config.json	3KB	MCP server configurations
lsp_config.json	2KB	LSP server configurations

Next Steps for AI Coding Agents

1. **Start with Phase 1** - Set up the infrastructure first
2. **Verify each phase** before moving to the next
3. **Use the detailed phase documents** for implementation specifics
4. **Follow the testing checklist** for validation
5. **Refer to generated code files** for working examples

Support & Troubleshooting

Common Issues

1. **RPC Connection Failed**

- Check firewall rules
- Verify IP addresses
- Test port connectivity

2. **Out of Memory**

- Reduce context size (-c parameter)
- Use smaller model
- Enable KV cache quantization

3. **Slow Inference**

- Verify GPU offloading
- Check network latency
- Optimize batch size

4. **Tool Execution Failed**

- Check MCP server health
- Verify tool parameters
- Review error logs

End of Master Implementation Guide